

Trabalho Avaliativo — Sistema de Recomendação por Afinidade

Objetivo

Desenvolver, em linguagem C, um sistema simples de recomendação capaz de identificar pessoas com perfis de interesse semelhantes a partir das notas atribuídas a diferentes preferências.

O trabalho será desenvolvido de forma evolutiva, em diferentes versões. Em todas elas, o problema e as funcionalidades principais deverão ser mantidos; o que mudará será a forma de organização dos dados na memória.

O objetivo é permitir a aplicação integrada dos seguintes conteúdos:

- matrizes;
- vetores paralelos;
- funções;
- busca e percursos;
- cálculo de distância euclidiana;
- alocação dinâmica de memória;
- estruturas (`struct`);
- vetores de estruturas;
- ponteiros;
- vetores de ponteiros para estruturas.

1. Contexto do problema

Uma plataforma deseja criar um recurso que permita identificar pessoas com interesses semelhantes.

Cada pessoa cadastrada deverá informar seu nome e atribuir uma nota de `0` a `10` para cada uma das seguintes preferências:

Código	Preferência
0	Música
1	Cinema
2	Jogos
3	Esportes
4	Leitura
5	Programação

Quanto mais semelhantes forem as notas atribuídas por duas pessoas, mais próximos serão seus perfis.

Para medir essa proximidade, o sistema utilizará a distância euclidiana.

2. Representação inicial dos dados

Na primeira versão do programa, os nomes das pessoas deverão ser armazenados em um vetor, enquanto suas notas deverão ser armazenadas em uma matriz.

Considere:

```
#define MAX_PESSOAS 30
#define QTD_PREF 6
```

Uma possível organização inicial será:

```
char nomes[MAX_PESSOAS][50];
float notas[MAX_PESSOAS][QTD_PREF];
```

Na matriz `notas`:

```
linha    → pessoa
coluna   → preferência
```

Assim:

PREFERÊNCIAS							
	0	1	2	3	4	5	
	MUS	CIN	JOG	ESP	LEI	PROG	
Pessoa 0 - Ana	9	8	3	2	9	8	
Pessoa 1 - Bruno	8	7	4	3	8	9	
Pessoa 2 - Carla	3	9	2	8	7	4	
Pessoa 3 - Diego	9	8	3	1	10	8	

Por exemplo:

```
notas[2][3]
```

representa a nota dada pela pessoa de índice `2` para a preferência de índice `3`.

3. Cálculo da semelhança entre duas pessoas

A proximidade entre duas pessoas deverá ser determinada por meio da distância euclidiana entre seus vetores de notas.

Se duas pessoas `A` e `B` possuem seis notas, a distância entre elas será calculada por:

$$d(A,B)=\sqrt{(\sum_{j=0 \text{ até } 5} (A_j - B_j)^2)}$$

Quanto menor for a distância, mais semelhantes são os perfis.

Observe:

```
DISTÂNCIA MENOR
↓
PERFIS MAIS PRÓXIMOS
↓
MAIOR SEMELHANÇA
```

Exemplo

Considere:

```
Ana   = [9, 8, 3, 2, 9, 8]
Bruno = [8, 7, 4, 3, 8, 9]
```

O cálculo será:

```
(9-8)2 +
(8-7)2 +
(3-4)2 +
(2-3)2 +
(9-8)2 +
(8-9)2

= 1 + 1 + 1 + 1 + 1 + 1

= 6
```

Portanto:

$$d(\text{Ana}, \text{Bruno}) = \sqrt{6} \approx 2,45$$

Se a distância entre Ana e outra pessoa for `1,41`, essa pessoa será considerada mais semelhante a Ana do que Bruno, pois:

```
1,41 < 2,45
```

4. Requisitos gerais

O programa deverá apresentar um menu semelhante ao seguinte:

```
=====
      SISTEMA DE AFINIDADES
=====

1 - Cadastrar pessoas
2 - Exibir pessoas e preferências
3 - Buscar pessoa pelo nome
4 - Comparar duas pessoas
5 - Encontrar pessoa mais semelhante
6 - Exibir ranking de afinidade
7 - Analisar preferências de duas pessoas
0 - Encerrar

Opcao:
```

O programa deverá ser modularizado em funções. Não será aceita uma solução em que todas as operações sejam realizadas diretamente na função `main()`.

5. ETAPA 1 — Matriz e estruturas paralelas

Na primeira implementação deverá ser utilizado:

```
char nomes[MAX_PESSOAS][50];
float notas[MAX_PESSOAS][QTD_PREF];
```

O aluno deverá perceber que:

```
nomes[i]
```

e

```
notas[i][j]
```

referem-se à mesma pessoa `i`.

5.1 Cadastro

O sistema deverá solicitar a quantidade de pessoas a serem cadastradas.

Nesta versão:

```
1 <= quantidade <= MAX_PESSOAS
```

Para cada pessoa deverão ser informados:

```
Nome
Nota para Música
Nota para Cinema
Nota para Jogos
Nota para Esportes
Nota para Leitura
Nota para Programação
```

Toda nota deverá estar no intervalo:

```
0 <= nota <= 10
```

Caso seja informado um valor inválido, o programa deverá solicitar novamente a nota.

Deverá existir uma função específica para validação da nota, por exemplo:

```
float leNotaValida(void);
```

5.2 Exibição dos dados

Os dados cadastrados deverão ser apresentados em formato tabular.

Exemplo:

```
=====
NOME      MUS   CIN   JOG   ESP   LEI   PROG
=====
Ana       9.0   8.0   3.0   2.0   9.0   8.0
Bruno     8.0   7.0   4.0   3.0   8.0   9.0
Carla     3.0   9.0   2.0   8.0   7.0   4.0
Diego     9.0   8.0   3.0   1.0  10.0   8.0
=====
```

5.3 Busca de uma pessoa

O usuário deverá informar um nome e o programa deverá procurar a pessoa cadastrada.

A busca deverá retornar:

```
índice da pessoa → caso encontrada
-1                → caso não encontrada
```

Sugestão de protótipo:

```
int buscaPessoa(char nomes[][50],
                int quantidade,
                char nomeProcurado[]);
```

6. ETAPA 2 — Distância entre duas pessoas

Deverá ser implementada uma função responsável por calcular a distância euclidiana entre duas pessoas.

Sugestão:

```
float calculaDistancia(float notas[][QTD_PREF],
                      int pessoa1,
                      int pessoa2);
```

A função deverá percorrer as seis preferências e realizar:

```
soma = 0

para cada preferência j:

    diferenca =
        notas[pessoa1][j] -
        notas[pessoa2][j]

    soma =
        soma + diferenca²

distancia = sqrt(soma)
```

A função deverá retornar a distância calculada.

6.1 Comparação entre duas pessoas

O usuário deverá informar duas pessoas cadastradas.

Exemplo:

```
Primeira pessoa: Ana
Segunda pessoa: Bruno
```

Saída:

```
=====
COMPARACAO DE PERFIS
=====

Ana x Bruno
```

Distancia euclidiana: 2.45

7. ETAPA 3 — Pessoa mais semelhante

O usuário deverá escolher uma pessoa como referência.

Exemplo:

Pessoa de referencia: Ana

O programa deverá comparar essa pessoa com todas as demais pessoas cadastradas.

A própria pessoa não poderá ser considerada candidata.

Portanto, se Ana possui índice `p`, não deverá ser calculada a distância:

Ana x Ana

pois:

$$d(\text{Ana}, \text{Ana}) = 0$$

e ela seria sempre o perfil mais próximo de si mesma.

Uma saída possível é:

```
=====
BUSCA DE PERFIS SEMELHANTES
=====

Pessoa de referencia: Ana

Distancia para Bruno: 2.45
Distancia para Carla: 10.86
Distancia para Diego: 1.41

-----
PERFIL MAIS SEMELHANTE
-----

Diego
Distancia: 1.41
```

O aluno deverá identificar que este é um problema de busca pelo menor valor.

8. ETAPA 4 — Ranking de afinidade

O programa deverá exibir todas as pessoas, exceto a pessoa de referência, ordenadas da menor distância para a maior distância.

Exemplo:

```
=====
PERFIS MAIS PROXIMOS DE ANA
```

```
=====
1 - Diego ..... 1.41
2 - Bruno ..... 2.45
3 - Carla ..... 10.86
```

Nesta primeira versão poderão ser utilizados vetores auxiliares, como:

```
float distancias[MAX_PESSOAS];
int indices[MAX_PESSOAS];
```

Os dois vetores deverão permanecer sincronizados.

Exemplo:

indices	distancias
3	1.41
1	2.45
2	10.86

9. ETAPA 5 — Análise detalhada das preferências

O programa deverá permitir comparar detalhadamente duas pessoas.

Exemplo:

```
=====
ANALISE DE PERFIS
=====

Ana x Diego

Preferencia      Ana      Diego      Diferenca
-----
Musica           9         9          0
Cinema           8         8          0
Jogos            3         3          0
Esportes         2         1          1
Leitura          9        10          1
Programacao      8         8          0
-----

Distancia euclidiana: 1.41
```

O sistema deverá também indicar a(s) preferência(s) em que as pessoas apresentam a menor diferença.

Exemplo:

```
Preferencias mais semelhantes:

Musica
Cinema
Jogos
Programacao
```

Caso exista empate, todas as preferências empatadas deverão ser apresentadas.

10. ETAPA 6 — Alocação dinâmica da matriz

A segunda versão completa do programa deverá eliminar o limite fixo de pessoas.

A quantidade de pessoas será conhecida somente durante a execução.

A matriz de notas deverá ser criada dinamicamente.

Nesta etapa deverá ser utilizada uma representação em bloco único de memória.

Exemplo conceitual:

```
float *notas;
```

A quantidade de memória deverá depender de:

```
quantidade de pessoas × quantidade de preferências
```

A matriz lógica continuará sendo:

	0	1	2	3	4	5
Ana	9	8	3	2	9	8
Bruno	8	7	4	3	8	9
Carla	3	9	2	8	7	4

mas será armazenada fisicamente de forma linear:

```
[9][8][3][2][9][8][8][7][4][3][8][9]...
```

O acesso a uma posição deverá considerar:

```
posição = linha × quantidade_de_colunas + coluna
```

O programa deverá:

- utilizar `malloc`;
- verificar se a alocação foi bem-sucedida;
- utilizar `sizeof`;
- liberar corretamente a memória com `free`.

Todas as funcionalidades das etapas anteriores deverão continuar funcionando.

11. ETAPA 7 — `struct Pessoa`

Na versão seguinte, os dados que representam uma pessoa deverão ser agrupados em uma estrutura.

Deverá ser definido um tipo semelhante a:


```
typedef struct {  
    char nome[50];  
    float notas[QTD_PREF];  
} Pessoa;
```

Observe a diferença conceitual.

Antes:

```
nomes[i]  
  
e  
  
notas[i][j]
```

Agora:

```
pessoas[i].nome  
  
pessoas[i].notas[j]
```

Cada posição do vetor passa a representar uma pessoa completa.

O programa deverá ser reescrito usando um vetor de `Pessoa`.

Todas as funcionalidades anteriores deverão ser preservadas.

12. ETAPA 8 — Vetor dinâmico de estruturas

A quantidade de pessoas continuará sendo definida em tempo de execução.

Consequentemente, o vetor de estruturas deverá ser criado dinamicamente.

A organização conceitual será:

```
pessoas  
|  
▼
```

```
Pessoa 0  
nome  
notas[0..5]
```

```
Pessoa 1  
nome  
notas[0..5]
```

```
Pessoa 2  
nome  
notas[0..5]
```

Deverão ser utilizados corretamente:

```
malloc  
sizeof  
free
```

13. ETAPA 9 — Vetor de ponteiros para `struct`

Na versão final, o programa deverá utilizar um vetor de ponteiros para estruturas.

O objetivo será permitir a criação do ranking sem alterar a ordem original do cadastro.

Considere:

```
CADASTRO ORIGINAL  
  
pessoas[0] = Ana  
pessoas[1] = Bruno  
pessoas[2] = Carla  
pessoas[3] = Diego
```

O ranking poderá armazenar endereços:

```
ranking[0] —————> Diego  
ranking[1] —————> Bruno  
ranking[2] —————> Carla
```

mesmo que o cadastro continue:

```
pessoas[0] = Ana  
pessoas[1] = Bruno  
pessoas[2] = Carla  
pessoas[3] = Diego
```

Nesta etapa deverão ser utilizados corretamente:

```
Pessoa *
```

e o operador:

```
->
```

O aluno deverá compreender que o ranking não precisa copiar todas as informações de uma pessoa. Ele pode guardar apenas o endereço do registro correspondente.

14. Funções

O trabalho deverá apresentar modularização adequada.

Não serão fornecidos todos os protótipos, pois parte da avaliação consiste em identificar corretamente:

- quais tarefas merecem uma função;
- quais informações a função precisa receber;
- qual deve ser o tipo de cada parâmetro;
- se a função deve ou não retornar um valor.

Entretanto, espera-se que existam funções relacionadas, no mínimo, às seguintes responsabilidades:

leitura e validação de notas cadastro exibição busca por nome cálculo da distância euclidiana localização do perfil mais semelhante construção do ranking análise das diferenças entre preferências
--

A função ``main()`` deverá atuar predominantemente como coordenadora das demais funções.

15. Requisitos obrigatórios

O programa deverá:

1. ser desenvolvido em linguagem C;
2. utilizar funções;
3. impedir notas fora do intervalo de ``0`` a ``10``;
4. utilizar uma matriz para representar as notas na primeira versão;
5. realizar busca pelo nome;
6. calcular corretamente a distância euclidiana;
7. identificar a pessoa mais próxima de uma pessoa escolhida;
8. não comparar a pessoa de referência consigo própria;
9. gerar um ranking por distância;
10. apresentar diferenças entre preferências;
11. utilizar alocação dinâmica na versão solicitada;
12. verificar falhas de ``malloc``;
13. utilizar ``free``;
14. utilizar ``struct Pessoa``;
15. implementar vetor de estruturas;
16. implementar vetor de ponteiros para estruturas;
17. preservar as funcionalidades à medida que o programa evolui.

16. Restrições

Não será permitido:

- utilizar variáveis globais para evitar a passagem de parâmetros;
- substituir a matriz por seis vetores independentes;
- escrever toda a solução dentro da ``main``;
- utilizar valores arbitrariamente grandes como substituição sistemática para a lógica correta de mínimo;
- ignorar a verificação de falha na alocação;
- omitir ``free`` nas versões com memória dinâmica;
- alterar a ordem original do cadastro apenas para produzir o ranking final.

17. Casos mínimos de teste

Antes da entrega, o grupo deverá testar pelo menos:

CASO 1
Duas pessoas com perfis idênticos
→ distância esperada = 0

CASO 2
Duas pessoas com perfis muito diferentes

CASO 3
Pessoa procurada não cadastrada

CASO 4
Nota menor que 0

CASO 5
Nota maior que 10

CASO 6
Empate entre duas pessoas na menor distância

CASO 7
Empate entre preferências com menor diferença

CASO 8
Pessoa de referência localizada na primeira posição

CASO 9
Pessoa de referência localizada na última posição

18. Entregas

O trabalho deverá ser entregue em versões sucessivas:

Versão	Estrutura utilizada
V1	vetor de nomes + matriz de notas estática
V2	representação com alocação dinâmica
V3	vetor de `struct Pessoa`
V4	vetor dinâmico de `struct Pessoa`
V5	vetor de ponteiros para `struct Pessoa`

Não deverá ser entregue apenas a versão final.

As versões anteriores fazem parte da avaliação, pois o objetivo do trabalho é justamente verificar a evolução da organização dos dados.

19. Relatório curto obrigatório

Junto ao código, deverá ser entregue um pequeno relatório respondendo, com as próprias palavras do grupo:

- a) Por que uma matriz é adequada para armazenar as notas na primeira versão?
- b) O que representa uma linha da matriz?
- c) O que representa uma coluna?
- d) Por que uma distância euclidiana menor indica maior semelhança?

e) Qual é a principal dificuldade de manter nomes e notas em estruturas paralelas?

f) Que problema a `struct Pessoa` resolve?

g) Qual é a diferença entre:

```
Pessoa pessoas[30];
```

e um vetor de pessoas criado com `malloc`?

h) Qual é a diferença conceitual entre:

```
Pessoa *p;
```

e:

```
Pessoa p;
```

i) Por que um vetor de ponteiros pode ser útil para gerar o ranking sem modificar a ordem do cadastro?

j) Em quais momentos do programa a memória dinâmica precisa ser liberada?

20. Critérios de avaliação

Eu sugiro pontuar o trabalho em 10,0 pontos, distribuídos assim:

Critério	Valor
V1 — matriz, cadastro, validação e busca	1,5
Cálculo correto da distância euclidiana	1,0
Pessoa mais semelhante	1,0
Ranking e análise das preferências	1,0
V2 — alocação dinâmica	1,5
V3/V4 — `struct` e vetor dinâmico de estruturas	1,5
V5 — vetor de ponteiros para `struct`	1,5
Modularização, legibilidade, testes e relatório	1,0
TOTAL	10,0

Observação importante

Uma versão posterior não substitui a implementação das anteriores. O objetivo central do trabalho é justamente observar a evolução:

```
MATRIZ + ESTRUTURAS PARALELAS
    ↓
  ALOCAÇÃO DINÂMICA
    ↓
  STRUCT PESSOA
    ↓
 VETOR DINÂMICO DE STRUCT
    ↓
VETOR DE PONTEIROS PARA STRUCT
```

Assim, a avaliação não considera apenas “se o programa funciona”, mas também se o estudante compreendeu como e por que a representação dos dados foi modificada ao longo do desenvolvimento.

Eu manteria esse último aspecto como parte importante da nota, porque é ele que transforma o trabalho de um exercício de programação em uma atividade realmente adequada à sequência didática de matrizes → memória dinâmica → `struct` → ponteiros.